

AURIX's Developer Role Guide

Role : IDE & Integration Engineer (VS Code)

Assigned to: DIVYANSHI

1. The "Why" of Your Component

Developers hate leaving their IDE to run security scans. Your component brings the power of AURIX directly to the developer's local machine. You are solving the hardest data-ingestion problem in the entire project: securely taking a developer's local, un-pushed, and messy codebase, packaging it up, and shipping it to the cloud for the AI to scan. You are bridging the gap between the user's local hard drive and our cloud architecture.

2. Your Domain in the Architecture (HLD & LLD References)

As the IDE Integration Engineer, you operate entirely on the client's local machine, but you must strictly follow the API contracts to communicate with the cloud:

- **HLD Ownership:** You own the **VS Code Extension** inside **Layer 1 (Client Interfaces)** of the [aurix-HLD.mmd](#) or [png](#) diagram. Like the Web Dashboard, you communicate *only* with Layer 2 (Bhavya's API Server).
- **LLD Ownership (API Flows):** You are the sole owner of the frontend logic detailed in the [Vscode-extension-api-sequence-diagram.mmd](#) or [png](#) . This is your master blueprint. You must build the logic to package the code and execute the POST /api/scans/upload request as a multipart/form-data payload.
- **LLD Ownership (Database & AI Link):** You do not touch the database directly, but your .zip payload is what Bhavya saves into the SCANS.storage_path column, and it is the exact file that Divyansh's AI Worker will download and scan. If your zip file is broken, the entire AI pipeline crashes.
- **LLD Ownership (Authentication):** Just like the Web UI, your VS Code extension must authenticate the user. You will need to build a small login prompt within VS Code, send the credentials to Bhavya's API, and securely store the returned JWT token using the VS Code SecretStorage API so the user doesn't have to log in every time they open their editor.

3. Step-by-Step Guide

1. **Scaffold the Extension:** Use the official Yeoman generator (yo code) to scaffold a TypeScript VS Code extension.
2. **Build the Auth Flow:** Create a command (e.g., Aurix: Login) that prompts the user for their email and password, hits Bhavya's login API, and saves the session token.
3. **Extension Settings & Configuration UI):** Create an aurix.settings block in your package.json so users can configure settings natively in the IDE, such as the aurix.apiEndpoint (switching between localhost/production) or aurix.scanTimeout.

4. **The "Zip & Ship" Logic (Your Biggest Task):** Write the local File System (fs) logic to recursively read the user's currently open workspace. You **must** parse their .gitignore file and use an archiving library to create a .zip file in memory or in a temp folder.
5. **Selective & Delta Scanning:** Implement logic that integrates with Git to allow smaller payload uploads. Give developers commands like Aurix: Scan Active File or Aurix: Scan Staged Changes to avoid zipping the whole project every time.
6. **Pre-Flight Privacy & Secret Guard:** Before zipping the files, run a fast local regex sweep for common hardcoded secrets (like AWS keys or un-ignored .env variables). If found, trigger a VS Code Warning Modal to block the upload until the developer confirms it is safe.
7. **API Integration:** Make the POST /api/scans/upload request, attaching the .zip file and the user's Auth token.
8. **Status Polling & Output Logs:** Implement the GET /api/scans/{scan_id} polling loop. Display a progress notification in the bottom right of VS Code (using vscode.window.withProgress). Simultaneously, stream process updates to a dedicated "AURIX" tab in the VS Code Output Panel for transparent logging and debugging
9. **Editor UI (The Results):** When the scan completes, parse the verified_vulnerabilities JSON payload. Use the VS Code Diagnostics API to underline the vulnerable code with red squiggly lines, and provide a "Hover" action or a Sidebar Panel to display the AI's logic_reasoning and the patch_code.
10. **"Quick Fix" Code Actions :** Implement the CodeActionProvider API. When a vulnerability is underlined, a yellow Lightbulb should appear allowing the user to click "AURIX: Apply AI Security Patch", which safely injects the suggested fix directly into the code.
11. **Dedicated "Attack Path" Webview Panel :** Build a rich HTML/React Webview Panel. When a user wants more than just a Hover tooltip, this panel opens inside the IDE to display the full, step-by-step exploit chain discovered by the AI Red Team.

4. Expected Output / Streamlining Flow

A compiled .vsix extension file that a user can install. Once installed, the developer should be able to click a button, watch a progress bar as their code is zipped and sent to the cloud, and within minutes, see AI-generated security patches pop up directly inside their code editor. Developers can securely block secret leaks, review rich attack paths natively, and apply code patches with a single click.

5. Critical "Do Not Miss" Constraints

- **The Node_Modules Nuke:** You **MUST** strictly enforce the .gitignore rules (especially ignoring folders like node_modules, venv, .git, and build). If your extension attempts to zip a massive node_modules folder, it will create a 500MB+ payload, freeze the user's computer, time out the HTTP request, and crash Bhavya's Backend server.
- **Preserve Folder Structure:** When you zip the files, the internal paths must be preserved (e.g., src/auth/login.js). If you flatten all the files into one folder, the AI Engine will not be able to find the correct files to patch.

- **Zero-Leak Guarantee** : Do not upload code to the cloud without passing the Pre-Flight Secret Guard checks.

6. Prerequisites & Skills Needed

- TypeScript and Node.js.
- Deep understanding of Node.js File System (fs, path) and Streams.
- Familiarity with the VS Code Extension API (Commands, Diagnostics, Webviews).

7. Reference Materials

Docs provided by tech leads

- [aurix-HLD.mmd](#) or [png](#)
- [Vscode-extension-api-sequence-diagram.mmd](#) or [png](#)
- [database-er-diagram.mmd](#) or [png](#) (for mapping your UI data).

External documentation

- [Your First VS Code Extension Guide](#)
- [npm archiver package \(for zipping in Node.js\)](#)
- [npm ignore package \(for accurately parsing .gitignore files\)](#)
- [VS Code API: SecretStorage \(for saving auth tokens\)](#)

8. Component Build List & Utility

- **VS Code Auth Module**: Keeps the developer securely logged in.
- **Workspace Packager**: Safely prepares and compresses local code for cloud transit.
- **API Connector**: The bridge between the local machine and the AURIX backend.
- **IDE Diagnostics UI**: Presents the AI findings natively to the developer via editor highlights and tooltips.
- **"Quick Fix" Auto-Patcher** : Applies AI code suggestions directly into the editor context.
- **Git-Aware Scanner**: Handles delta scans (active file/staged changes) to save bandwidth.
- **Pre-Flight Secret Guard**: Blocks accidental uploads of local API keys or .env files.
- **Configuration Manager**: Handles aurix.settings for dynamic API endpoints and custom ignores.
- **Attack Path Webview** : Renders rich, HTML-based exploit chain reports inside VS Code.
- **AURIX Output Logger** : Maintains a transparent activity and error log for debugging.

9. CRITICAL DEPENDENCIES (Blockers)

- **You are BLOCKED BY the Backend Engineer (Bhavya)**: Just like Bhumika, you cannot build your upload or polling logic without the API. You must ask Bhavya for "Mock Endpoints" immediately so you can test your Zip upload logic against a fake server before the real database is ready.
- **You BLOCK the Core Engine Architect (Divyansh)**: Your .zip format is the exact payload Divyansh's AI needs to read. You two must coordinate to ensure that when he unzips your

file in his Docker sandbox, the file paths exactly match what the static analyzer expects.